DTU Compute

02428
September 1, 2026
uhth

Exercises for session 1: Static optimization

Constrained and unconstrained optimization in the plane

Consider the following problem:

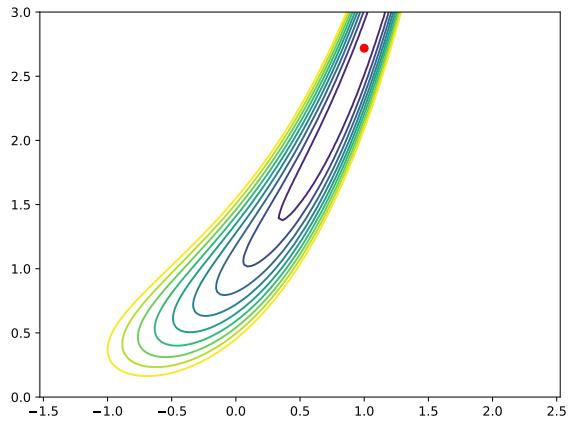
$$\min_{x,y} h(x,y) \text{ where } h(x,y) = (x-1)^2 + b(y-e^x)^2.$$

Here, $b = 10$ is a parameter.

Question 1: Identify analytically the minimum point. Plot a contour plot of the function and mark the minimum point.

Solution: It is easy to see that $h(x,y) \geq 0$ and that $h(x,y) = 0$ if and only if $x = 1$ and $y = e^x$, i.e., $y = e$. The following code makes the contour plot.

```
import numpy as np
import matplotlib.pyplot as plt

b = 10

def h(x, y):
  return (x-1) ** 2 + b*(y-np.exp(x))**2

x, y = np.meshgrid(np.linspace(-1, 2,100),np.linspace(0, 3, 100))

plt.contour(x, y, h(x, y),np.linspace(0,4,10))
plt.axis('equal')
plt.tight_layout()
plt.plot(1,np.exp(1),'ro')
plt.savefig ("contour.pdf")
```



Question 2: Solve the optimization problem numerically using your favorite optimizer.

Solution: This can be done in many ways; the following uses casadi with ipopt from python.

```
import casadi
import pylab as pl
```

```
opti = casadi.Opti()

x = opti.variable()
y = opti.variable()

z = h(x,y)

opti.minimize(z)

opti.solver('ipopt')
solUnc = opti.solve()
```

The code finds the optimum: $x = 1.0000$, $y = 2.7183$.

Question 3: Solve the optimization problem numerically, subject to an equality constraint. That is, define for $s > 0$

$$J(s) = \min\{h(x, y) | x^2 + y^2 = s\}.$$

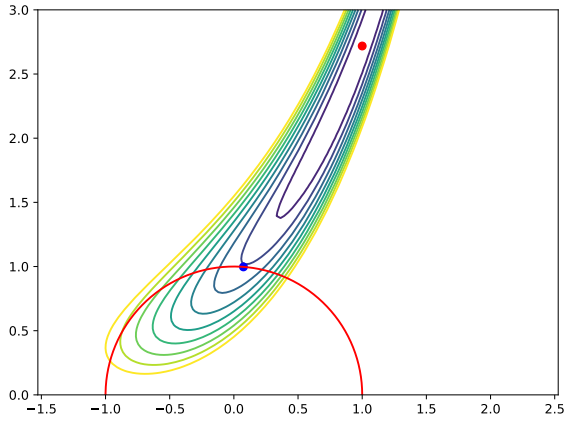
Determine $J(1)$ numerically. Identify the associated Lagrange multiplier μ. Verify numerically that

$$J(1 + \epsilon) \approx J(1) - \mu\epsilon,$$

taking, e.g., $\epsilon = 10^{-2}$.

Solution: We add a constraint to the problem and solve the constrained problem. We plot the constraint (a circle) and the constrained minimum point.

```
s = opti.parameter()
s0 = 1
opti.set_value(s,s0)

con = x**2+y**2==s
opti.subject_to(con)

solCon = opti.solve()

plt.plot(solCon.value(x),solCon.value(y),'bo')
tc = np.linspace(0,np.pi,200)
xc = np.sqrt(s0)*np.cos(tc)
yc = np.sqrt(s0)*np.sin(tc)
plt.plot(xc,yc,'r-')
plt.savefig('constrained.pdf')
plt.close()
```



Next, we compute the Lagrange multiplier; this we get from the optimized object. We also perturb the constraint slightly, recompute the optimum, and check that change in the value agrees with the Lagrange multiplier.

```
zopt = solCon.value(z)
mu = solCon.value(opti.dual(con))

eps = 1e-2

opti.set_value(s,1 + eps)
solPert = opti.solve()

zpert = solPert.value(z)
```

We find $\mu = 0.8052$. The values of the two optimization problems are 0.9207 and 0.9128, respectively; the difference -0.0080 agrees with $\epsilon\mu = 0.0081$.

A parametric program

We consider an optimization problem over $u \in [0,1]$ in which a parameter x enters. Specifically, consider the function $J : \mathbf{R} \mapsto \mathbf{R}$ given by

$$J(x) = \min_{u\in[0,1]} f(x,u) \text{ where } f(x,u) = (x-u)^2.$$

Also, define the optimizing u:

$$\mu(x) = \text{Arg} \min_{u\in[0,1]} f(x,u).$$

Question 4: Compute $J(x)$ and $\mu(x)$ and sketch or plot them. Verify that

$$J'(x) = \frac{\partial f}{\partial x}(x, \mu(x)).$$

Optional: Show that this last result holds not just for this particular parametric program, but in generality under weak regularity conditions.

Solution: Ignore first the constraint; then the optimum would be found at the stationary point, $u = x$. Now, with the constraint, there are three situations:

1. $x \in [0,1]$: Then $\mu(x) = x$.
2. $x < 0$: Then the optimum is found at the left boundary, $\mu(x) = 0$.
3. $x < 1$: Then the optimum is found at the right boundary, $\mu(x) = 1$.

In summary, $\mu(x)$ is the projection of x on $[0,1]$. The resulting value function is

$$J(x) = \begin{cases} 0 & \text{when } x \in [0,1] \\ x^2 & \text{when } x < 0 \\ (x-1)^2 & \text{when } x > 1. \end{cases}$$

which is the squared distance of x to the set $[0,1]$.

It is straightforward to verify the relationship between $J'(x)$ and $\partial f/\partial x$. The reason it holds in general is that

$$J' = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial u}\nabla\mu.$$

Briefly, the optimum can either be at a stationary point, $\partial f/\partial u = 0$, or a boundary point, in which case $\nabla\mu = 0$. In either case, we get $J' = \partial f/\partial x$.

Completion of the square

Consider the quadratic functional in x, y

$$f(x,y) = \frac{1}{2}(x^\top \; y^\top) \begin{bmatrix} A & B \\ B^\top & C \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

where $x \in \mathbf{R}^n$, $y \in \mathbf{R}^m$, and the matrices have compatible dimensions, so e.g. $A \in \mathbf{R}^{n\times n}$. We take $A = A^\top$ and $C = C^\top > 0$.

Question 5: Determine analytically

$$g(x) = \min_y f(x,y)$$

as well as the minimizing argument y as a function of x. Finally, determine $f(x,y) - g(x)$.

Solution: For given x, the function $y \mapsto f(x,y)$ is strictly convex, so the minimum is found by identifying the stationary point:

$$B^\top x + Cy = 0 \quad y = -C^{-1}B^\top x.$$

Inserting, this gives

$$g(x) = \frac{1}{2}x^\top(A - BC^{-1}B^\top)x.$$

We can write $f(x,y)$ as

$$f(x,y) = \frac{1}{2}x^\top(A - BC^{-1}B^\top)x + \frac{1}{2}|C^{1/2}y + C^{-1/2}B^\top x|^2$$

so

$$f(x,y) - g(x) = \frac{1}{2}|C^{1/2}y + C^{-1/2}B^\top x|^2.$$

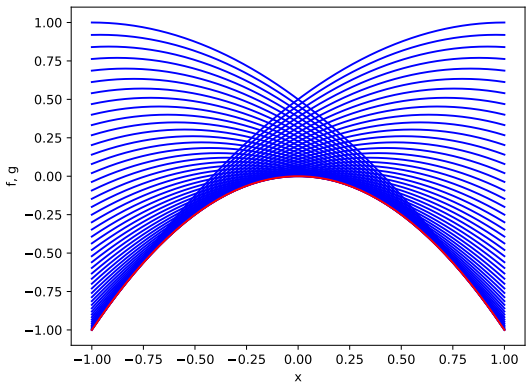
Question 6: Consider $A = -1$, $B = 1$, $C = 1$. Plot, as a function of x, $f(x,y)$ for 20 suitably chosen values of y. Include $g(x)$ in the plot.

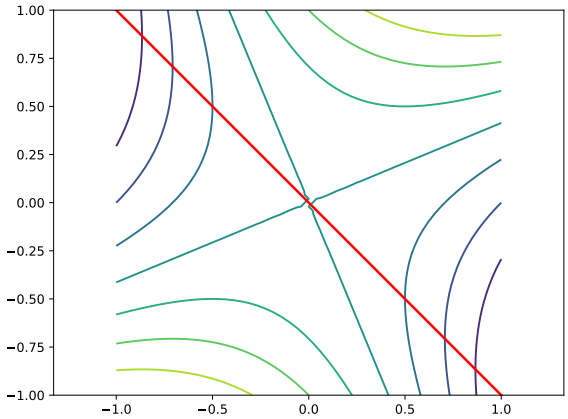
Plot also a contour plot of $f(x,y)$ and indicate, for each x, the optimal y.

Solution: The following code solves the question:

```
import matplotlib.pyplot as plt
import numpy as np

yv = np.linspace(-1,1)

xv = np.linspace(-1,1)

A = -1
B = 1
C = 1

def f(x, y):
  return 0.5*A*x**2+x*B*y+ 0.5 * y**2

def g(x):
  return 0.5*x**2*(A-B**2/C)

for i in np.arange(yv.size):
    plt.plot(xv,f(xv,yv[i]),'b')
```

```
plt.plot(xv,g(xv),'r')
plt.xlabel('x')
plt.ylabel('f, g')
plt.savefig ("quadratics.pdf")
plt.close()

x, y = np.meshgrid(xv,yv)

plt.contour(x, y, f(x, y))
plt.axis('equal')
plt.tight_layout()
plt.plot(xv,-B*xv,'r',linewidth=2)
plt.savefig ("quad-contour.pdf")
```





Note, in the contour plot, that for each x, the optimal y (given by the red line) is always found where the contour lines are vertical, indicating $\partial f/\partial y = 0$.

Question 7: Generate random matrices $A = A^\top$, B and $C > 0$ for, say, $n = 20$ and $m = 20$; use a probability distribution of your own choice. Solve the optimization problem $\min_y f(x, y)$ numerically and verify that the solution agrees with analytical result. Changing the dimension or the distribution, do some quick experimentation to gain some experience with when the numerical solution agrees.

Solution: The following code solves the problem. The matrices are generated based on Gaussians; with A, we add the transpose to ensure that the result is symmetric, while with C, we multiply with the transpose and add a little in the diagonal to ensure that the result is positive definite and does not have a too bad condition number. We use the built-in optimizer `scipy.optimize.minimize`.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
n = 20
m = 20

A = np.random.normal(size=(n,n))
A = A+A.transpose()

B = np.random.normal(size=(n,m))
C = np.random.normal(size=(m,m))
C= C @ C.transpose() + 0.01*np.eye(m)

from scipy.optimize import minimize
x = np.random.normal(size=(n))

M = np.block([[A,B],[B.transpose(),C]])

def quad(y):
    xy = np.r_[x,y]
    return np.dot(xy,M @ xy)

opt = minimize(quad,np.random.normal(size=(m)))

yopt = opt.x
ysol = -np.linalg.solve(C,B.transpose() @ x)


err = np.linalg.norm(yopt-ysol)
```

For this example and this realization, we get an error of 0.0003, which seems OK but is not super impressive. Larger problems, and in particular ones with worse condition number, do worse. With this code, without the diagonal added to C, the error is random and OK most of the time but sometimes really bad. Perhaps `casadi` would have done better?

Pareto optimality

The following is a simple example where two conflicting objectives must be traded off: An investor can buy several assets (e.g., stocks) and is interested in a large return. However, returns are random, and assets with high expected return also have high variance. The investor wants to maximize the expected return, but also to minimize the variance of the return.

Let X_i be independent random variables, each distributed as $X_i \sim N(i, i)$, for $i = 1, 2, 3$. We form a convex combination of these variables:

$$Y(w) = \sum_{i=1}^{3} w_i X_i$$

where $w \in \mathbf{R}^3$ are weights such that $w_i \geq 0$, $\sum_i w_i = 1$.

Now, we have the two objectives, which ideally should both be small: The expected loss

$$J_1(w) = -\mathbf{E}Y(w),$$

and the variance on the loss

$$J_2(w) = \mathbf{V}Y(w).$$

Question 8: Find the w that minimizes $J_1(w)$. Next, find the w that minimizes $J_2(w)$.

Solution: It is easy to see that $J_1(w)$ is minimized by investing all money in the asset which we expect to perform best, i.e., $w = (0, 0, 1)$. We minimize $J_2(w)$ numerically:

```
import casadi
import matplotlib.pyplot as plt
import pylab as pl

## Number of risky assets
n = 3

## Mean and variance on return from each asset
mu = pl.linspace(1,n,n)
s2 = pl.linspace(1,n,n)

## Set up optimization problem
opti = casadi.Opti()

w = opti.variable(n)
theta = opti.parameter()  ## Use this to trade-off the two J's

## Constraints on the weights w
SignCon = w >= 0
SumCon = casadi.sum1(w)==1

## The two objectives: Mean and variance
J1 = -casadi.sum1(w*mu)
J2 =  casadi.sum1(w**2 * s2)

## Combined objective
J = theta*J1 + (1-theta)*J2

## First solve the problem for J2
opti.set_value(theta,0.)

opti.minimize(J)
opti.subject_to(SignCon);
opti.subject_to(SumCon);

opti.solver('ipopt')
sol = opti.solve()
```

The result is $w = (0.5455, 0.2727, 0.1818)$. Note that most money is investment in the asset with the least variance.

Question 9: Determine the Pareto front. Specifically, for various values of $\theta \in [0, 1]$, solve the problem

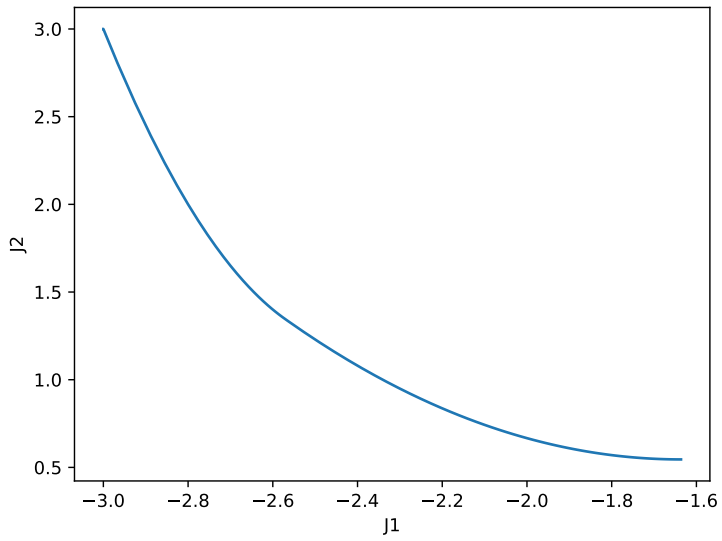
$$\min_w [\theta J_1(w) + (1 - \theta) J_2(w)]$$

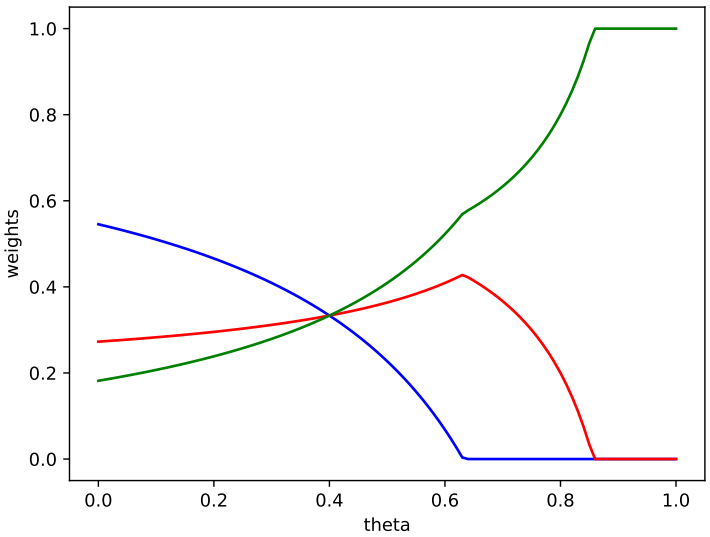
numerically, subject to the constraints given. Plot the front in the (J_1, J_2)-plane. Plot also the optimal weights (w_1, w_2, w_3) as functions of θ. Can you explain qualitatively the shape of the curves between the end points?

Solution:

```
## Parametric sweep to locate the Pareto front
nth = 101
ths = pl.linspace(0,1,nth)
J1s = pl.zeros(nth)
```

```
J2s = pl.zeros(nth)
Ws = pl.zeros((nth,n))

for i in range(nth):
    opti.set_value(theta,ths[i])
    sol = opti.solve();
    J1s[i] = sol.value(J1)
    J2s[i] = sol.value(J2)
    Ws[i,:] = sol.value(w)

## Plot the front
plt.figure()
plt.plot(J1s,J2s)
plt.xlabel('J1')
plt.ylabel('J2')
plt.savefig('front.pdf')
plt.close()

## Plot the optimal weights
plt.figure()
plt.plot(ths,Ws[:,0],'b')
plt.plot(ths,Ws[:,1],'r')
plt.plot(ths,Ws[:,2],'g')
plt.xlabel('theta')
plt.ylabel('weights')
plt.savefig('weights.pdf')
plt.close()
```





To interpret these curves, recall first that $\theta = 0$ corresponds to minimizing the variance, while $\theta = 1$ corresponds to maximizing the mean. For small θ, we hedge the bets. As θ is increased, we weight the high-gain high-risk asset more, while downplaying asset 1.

Note that the curves are only piecewise smooth; corners appear when the optimal weight hits the boundary of the feasible set.

As for the Pareto front, it is clear that higher expected gains (J_1) come with higher variance (J_2). It is also clear that front should be convex and piecewise smooth.

Maximizing the effect of refraction

Question 10: A thin straw is placed vertically in a circular cylindrical glass with radius 1, which is half full of water. When viewed from the side, it appears that the upper and lower parts of the straw are a distance d apart; see the figure. Choose the position (x, y) of the straw which maximizes the distance d. Use a refractive index of 1.33 for water.

Note: This exercise can be approached in many different ways and it is worthwhile to at least consider the options.

Solution: It is clear that the point (x, y) should be chosen on the boundary, $x^2 + y^2 = 1$. To see this, note in the figure that we could move the point (x, y) back, extending the blue line inside the glass. This would maintain the angles but increase the distance d. Let $x = \cos\phi$, $y = \sin\phi$, where y is upwards in the figure, and let the point where the ray of light exits the water be $u = \cos\theta$, $v = -\sin\theta$. We can therefore restate the problem as

$$\max(\cos\theta - \cos\phi)$$

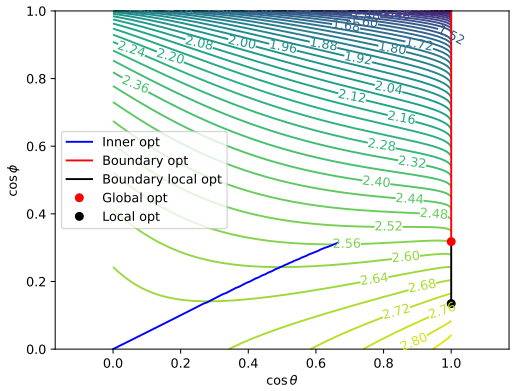
subject to the constraint that for given ϕ, θ minimizes the travel time T to the line $y = -1$:

$$T(\theta, \phi) = n\sqrt{(\cos\theta - \cos\phi)^2 + (\sin\theta + \sin\phi)^2} + 1 - \sin\theta.$$

We require $\theta \in [0, \pi/2]$ - clearly, the image must be on the side facing the observer, and by symmetry, we only consider image on the right side. We only consider $\phi \in [0, \pi/2]$ - one could argue that a value $\theta \in [0, \pi/2]$ can be a local minimum point for $\phi \in [\pi/2, \pi]$, but we disregard this option. The following contour plot shows T as a function of $\cos\theta$ and $\cos\phi$. The plot includes the value of θ which minimizes the time, for each ϕ, and also if the boundary is a local optimum.

```
import matplotlib.pyplot as plt
import numpy as np
# from scipy import optimize
# import itertools

n = 1.33

## Travel distance
def Time(phi,theta):
    return ( np.sqrt((np.cos(phi)-np.cos(theta))**2 + (np.sin(phi)+np.sin(theta))**2)*n + 1. - np.sin(t

phis = np.linspace(0,np.pi/2,501)
thetas = np.linspace(0,np.pi/2,499)

TT = np.array(
    [[Time(phis[i],thetas[j]) for j in range(len(thetas))] for i in range(len(phis))]
)

fig = plt.figure()
CS = plt.contour(np.cos(thetas),np.cos(phis),TT,levels = 50)
plt.xlabel(r'$\cos\theta$')
plt.ylabel(r'$\cos\phi$')
ax = fig.gca()

ax.axis('equal')

ax.clabel(CS, CS.levels)

## For each phi, find optimal theta
thmin = TT.argmin(axis = 1)

## Identify when the optimum is at an inner point
Inner_opt = thmin > 0

## ... and when the boundary is a local optimum
Local_Boundary = (thmin > 0 ) & (TT[:,0] < TT[:,1])

## Find the global optimum
distances = np.cos(thetas[thmin]) - np.cos(phis)
I = distances.argmax()

## Allow for local optima as well
Iloc = len(Local_Boundary) - 1 - np.argmax(Local_Boundary[::-1])

## Mark these points in the plot
ax.plot(np.cos(thetas[thmin[Inner_opt]]),np.cos(phis[Inner_opt]),'b',label="Inner opt")
ax.plot(np.cos(thetas[thmin[~Inner_opt]]),np.cos(phis[~Inner_opt]),'r',label="Boundary opt")
ax.plot(np.cos(0*thetas[thmin[Local_Boundary]]),np.cos(phis[Local_Boundary]),'k',label="Boundary local
```

```
ax.plot(1.,np.cos(phis[I]),'ro',label="Global opt")
ax.plot(1.,np.cos(phis[Iloc]),'ko',label="Local opt")

ax.legend()


plt.savefig('Refraction-contour.pdf')
plt.close()
```



To maximize the distance, we look for the point in the plot which is as far down towards the southeast corner as possible. We see that we have $\cos\theta = 1$, i.e., $\theta = 0$. Requiring that the value $\theta = 0$ is a global minimizer of the travel time, then we get the red point in the graph, which has a numeric value $\cos\phi = 0.31$ leading to a distance $d = 0.69$.

Note: If we are satisfied with the value $\theta = 0$, being a local minimizer, then we get the black point. Along the black line, the pencil will appear to be in two places. However, the travel time is a very flat function of θ for ϕ in this range, which means that slight imperfections in the model can have large consequences - for example, the effect of the glass.

The following photo illustrates the situation.

Convexity

Question 11: Which of the optimization problems considered in the previous are convex? Does it matter?

Solution: The function $h(x, y) = (x - 1)^2 + b(y - e^x)^2$ is not convex.

For the geodesics on the sphere, the cost function is convex, but the feasible set is not convex, so the optimization problem is not convex.

The parametric program is a convex optimization problem in u, for each x.

When completing the squares, the inner problem (minimize over y) is convex.

In the Pareto problem, the cost functional is convex in the weights, and the feasible set is convex, so the optimization problem is convex.

The refraction problem is not convex.

The lack of convexity of the two first problems did not cause severe problems. In both cases, we were able to give good initial guesses, and the problems were simple enough that we did not worry about multiple local minima. In the refraction problem, the lack of convexity is connected to the singularities and possible multiple local minima, which all add to the complexity.

For more complex problems, which are more difficult to visualize and perhaps are badly scaled, convexity is helpful: Numerical algorithms are better at finding local minima in the convex case, and we know that the result is then a global optimum.